

PLATFORM OWNER (SUPER ADMIN) BACKEND SPECIFICATION DOCUMENT

EduManage School Management System

Software Requirements Specification (SRS)

Version Information

Document Title: Platform Owner (Super Admin) Backend Specification Document

System Name: EduManage School Management System

Module: Platform Owner (Super Admin)

Document Type: Software Requirements Specification (SRS)

Version: 1.0

Status: Final Draft

Prepared By: _____

Reviewed By: _____

Approved By: _____

Institution/Organization: _____

Date: _____

CHAPTER ONE

INTRODUCTION

1.1 Purpose of the Document

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Platform Owner (Super Admin) module of the EduManage School

Management System. The document serves as a comprehensive technical guide for backend developers, software architects, database designers, system administrators, quality assurance teams, and project stakeholders involved in the implementation, testing, deployment, and maintenance of the module.

The Platform Owner module represents the highest level of system administration within the EduManage platform. It provides centralized control over all schools registered on the platform and enables platform-wide management of subscriptions, billing, user administration, analytics, system monitoring, developer tools, support services, configuration settings, and emergency controls.

This specification translates the approved frontend design into detailed backend requirements by defining the expected system behaviour, business logic, database interactions, API requirements, validation rules, security mechanisms, and processing workflows required to implement each Platform Owner function.

1.2 Scope

This document covers the backend implementation requirements for all functionalities available to the Platform Owner (Super Admin). The scope includes the services, APIs, business rules, database requirements, access control mechanisms, validation procedures, logging, auditing, notification handling, and system integrations necessary to support the Platform Owner interface.

The specification encompasses the following functional areas:

- Platform Overview Dashboard
- Schools Management
- Subscription and Billing Management
- Platform Analytics
- Global Configuration Management
- Platform Administrator Management
- Support and Helpdesk Management
- System Monitoring
- Developer Tools
- Emergency Controls

The document focuses exclusively on backend behaviour and system functionality. While references are made to frontend components where necessary to explain system interactions, the design, styling, and presentation of the user interface are outside the scope of this specification.

1.3 Objectives

The primary objectives of this specification are to:

- Provide a complete backend implementation guide for the Platform Owner module.

- Define functional requirements for every Platform Owner page and feature.
- Establish standard API requirements for communication between the frontend and backend.
- Define database entities and relationships supporting Platform Owner operations.
- Specify validation rules for all system transactions.
- Define authentication and authorization requirements.
- Establish security and auditing standards.
- Ensure consistency across all backend services.
- Provide a reference document for software testing and quality assurance.
- Support future system maintenance, upgrades, and scalability.

1.4 Intended Audience

This document is intended for:

- Backend Software Developers
- Database Designers and Administrators
- Software Architects
- Frontend Developers requiring API integration
- Quality Assurance and Software Testing Teams
- DevOps and Infrastructure Engineers
- Project Managers
- System Administrators
- Technical Review Committees
- Future Development Teams responsible for system maintenance and enhancement

1.5 System Overview

EduManage is a centralized multi-tenant School Management System designed to enable multiple educational institutions to operate independently while sharing a common software platform. Each school maintains its own users, academic records, financial information, communication channels, and administrative processes within an isolated tenant environment.

The Platform Owner operates above all tenant schools and possesses unrestricted administrative privileges over the platform. Responsibilities include onboarding schools, managing subscriptions, monitoring platform performance, administering platform-wide settings, overseeing support operations, maintaining system integrity, and ensuring the availability, security, and reliability of the platform.

The Platform Owner module acts as the operational control centre for the entire EduManage ecosystem by providing visibility into platform usage, operational health, user activities, and system-wide performance metrics.

1.6 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface used for communication between the frontend and backend services.
Authentication	The process of verifying the identity of a user before granting access to the system.
Authorization	The process of determining what resources and operations an authenticated user is permitted to access.
Backend	The server-side components responsible for business logic, data processing, and communication with the database.
Dashboard	A centralized interface displaying summarized information and operational statistics.
Database	A structured repository used for storing and managing system data.
Platform Owner	The highest privileged administrator responsible for managing the entire EduManage platform.
REST API	Representational State Transfer Application Programming Interface using standard HTTP methods.
SRS	Software Requirements Specification.
Super Admin	Another term used to describe the Platform Owner with unrestricted platform privileges.
Tenant	An individual school operating independently within the shared EduManage platform.

1.7 Assumptions

The following assumptions apply throughout this specification:

- The frontend user interface has been finalized and approved.
- Backend services will follow RESTful API principles.
- All data will be stored within a relational database management system.
- Every API request requiring protected resources shall require authentication.
- Role-Based Access Control (RBAC) shall govern authorization decisions.
- All sensitive operations shall generate audit logs.
- Communication between clients and servers shall occur over secure HTTPS connections.
- Platform services shall support future scalability without requiring significant architectural changes.

1.8 Constraints

The backend implementation shall comply with the following constraints:

- All Platform Owner operations shall be accessible only to authorized Platform Owner accounts.
- The system shall prevent unauthorized access through authentication and authorization mechanisms.
- Database transactions involving critical operations shall maintain atomicity and consistency.
- Sensitive information shall be encrypted during transmission and securely stored where applicable.
- All business operations shall be logged for auditing and accountability.
- APIs shall maintain backward compatibility where practical to support future enhancements.

1.9 Document Organization

This Software Requirements Specification is organized into eight chapters:

- **Chapter One:** Introduction
- **Chapter Two:** Platform Owner Module Overview
- **Chapter Three:** Functional Page Specifications
- **Chapter Four:** Backend Architecture
- **Chapter Five:** Database Design
- **Chapter Six:** API Standards and Specifications
- **Chapter Seven:** Security Requirements
- **Chapter Eight:** Acceptance Criteria, Testing, and Implementation Considerations

Each chapter builds upon the previous one, progressing from high-level concepts to detailed technical requirements. Together, they provide a complete specification for designing, implementing, testing, deploying, and maintaining the Platform Owner backend module of the EduManage School Management System.

CHAPTER TWO

PLATFORM OWNER MODULE OVERVIEW

2.1 Introduction

The Platform Owner (Super Admin) Module is the highest-level administrative component of the EduManage School Management System. It provides centralized management and governance of

the entire multi-tenant platform, enabling the Platform Owner to oversee all registered schools, administrators, subscriptions, system resources, platform configurations, operational analytics, and platform health from a single administrative interface.

Unlike school-level administrative modules, which operate within the boundaries of an individual institution, the Platform Owner module operates globally across the platform. It is responsible for ensuring that every subsystem functions correctly, securely, and efficiently while maintaining data isolation between schools.

The Platform Owner module acts as the command center of the EduManage ecosystem and provides the authority required to monitor, configure, maintain, and recover the platform when necessary.

2.2 Role of the Platform Owner

The Platform Owner serves as the overall administrator of the EduManage platform and possesses unrestricted privileges across all system tenants. This role is responsible for the strategic administration, operational monitoring, security enforcement, and technical maintenance of the entire platform.

The Platform Owner performs responsibilities including:

- Registering and managing schools.
- Monitoring subscription plans and billing.
- Managing platform administrators.
- Configuring global platform settings.
- Monitoring platform performance.
- Managing customer support operations.
- Accessing developer maintenance utilities.
- Performing emergency recovery operations.
- Reviewing platform-wide analytics and reports.
- Maintaining overall platform security and availability.

The Platform Owner does not participate in the day-to-day academic or administrative activities of individual schools. Instead, the role focuses on maintaining the platform that enables schools to operate effectively.

2.3 System Architecture Overview

EduManage follows a multi-tier architecture consisting of presentation, application, data, and infrastructure layers. Each layer performs distinct responsibilities while communicating through well-defined interfaces.

Presentation Layer

The Presentation Layer consists of the React-based frontend application used by Platform Owners. It provides dashboards, management pages, reports, configuration interfaces, monitoring tools, and administrative controls. User interactions from this layer are transmitted to the backend through secure REST APIs.

Application Layer

The Application Layer contains the backend services responsible for implementing business logic. These services process requests, validate user input, enforce business rules, execute workflows, communicate with external services, and interact with the database.

Major backend services include:

- Authentication Service
- School Management Service
- Subscription Service
- Billing Service
- Analytics Service
- Configuration Service
- Administrator Management Service
- Support Service
- Monitoring Service
- Developer Service
- Emergency Management Service
- Notification Service
- Audit Logging Service

Data Layer

The Data Layer stores persistent system information using a relational database management system. It maintains platform configuration, school records, user accounts, subscriptions, billing transactions, audit logs, monitoring data, reports, and operational statistics.

Data integrity is maintained through relational constraints, validation rules, transactions, indexing, and backup mechanisms.

Infrastructure Layer

The Infrastructure Layer provides hosting, networking, security, monitoring, storage, backups, logging, caching, and deployment services required for reliable system operation.

2.4 Multi-Tenant Architecture

EduManage is designed as a multi-tenant Software-as-a-Service (SaaS) platform, allowing multiple schools to share a common application while maintaining complete logical separation of institutional data.

Each school functions as an independent tenant with its own:

- Students
- Staff
- Academic records
- Financial records
- Attendance data
- Communication records
- Timetables
- Examination information
- Reports
- User accounts

The Platform Owner operates above all tenants and can view or manage platform-level information without compromising tenant isolation. Backend services must enforce tenant boundaries for school-level users while allowing authorized Platform Owner operations where necessary.

2.5 Functional Responsibilities

The Platform Owner module consists of several integrated functional components that collectively support centralized administration.

Platform Overview

Provides a real-time summary of platform activities, system health, active schools, subscription statistics, revenue indicators, notifications, and operational alerts.

Schools Management

Supports onboarding, updating, suspension, activation, verification, and lifecycle management of registered schools.

Subscription and Billing

Manages subscription plans, invoices, payment records, renewals, billing cycles, pricing models, and revenue monitoring.

Analytics

Generates operational reports and visual analytics covering user growth, platform utilization, financial performance, school activity, and system usage trends.

Configuration

Provides centralized management of global platform settings, branding, communication preferences, feature toggles, integrations, localization, and default system behavior.

Platform Administrator Management

Supports creation, modification, activation, deactivation, and permission management of Platform Administrator accounts while maintaining role-based access control.

Support and Helpdesk

Manages customer support tickets, issue tracking, escalation procedures, knowledge base resources, and communication history.

System Monitoring

Monitors application availability, server health, resource utilization, background processes, error logs, and performance indicators.

Developer Tools

Provides technical maintenance utilities including cache management, scheduled job execution, API diagnostics, log inspection, testing tools, and maintenance operations.

Emergency Controls

Supports disaster recovery, maintenance mode, emergency shutdown, backup restoration, rollback operations, and incident response procedures.

2.6 User Roles and Permissions

Although the Platform Owner module primarily serves the Platform Owner, controlled access may also be granted to designated Platform Administrators based on assigned permissions.

The backend shall implement Role-Based Access Control (RBAC) to enforce authorization.

Typical permission categories include:

- School Management
- Billing Management
- User Administration
- Platform Configuration
- Analytics Access
- Monitoring Access
- Developer Utilities
- Support Operations
- Emergency Controls
- Audit Log Access

Every request shall be validated against the authenticated user's assigned permissions before processing.

2.7 Integration with Other Modules

The Platform Owner module communicates with multiple backend services across the EduManage platform.

Key integrations include:

- Authentication Module
- School Administration Module
- Finance Module
- Student Information Module
- Human Resource Module
- Communication Module
- Notification Service
- Payment Gateway
- Email Service
- SMS Gateway
- Audit Logging Service
- File Storage Service
- Reporting Engine

These integrations enable coordinated platform operations while maintaining modular system architecture.

2.8 Data Flow Overview

A typical backend workflow follows these stages:

1. The Platform Owner submits a request through the frontend interface.
2. The request is authenticated using a secure authentication mechanism.
3. Authorization checks verify the user's permissions.
4. Input validation ensures all submitted data meets defined constraints.
5. The appropriate backend service processes the request.
6. Required database transactions are executed.
7. Audit logs are generated for traceability.
8. Notifications are triggered where applicable.
9. A standardized API response is returned to the frontend.

This workflow ensures consistency, reliability, accountability, and security across all Platform Owner operations.

2.9 Non-Functional Requirements

The Platform Owner backend shall satisfy the following non-functional requirements:

Performance

The system shall provide responsive API execution for normal administrative operations and efficiently handle concurrent requests without significant degradation in performance.

Scalability

The architecture shall support the addition of new schools, users, modules, and services without requiring major structural redesign.

Availability

The platform shall be designed to maximize uptime through redundancy, health monitoring, automated recovery mechanisms, and backup procedures.

Reliability

Critical operations shall use transactional processing to ensure data consistency and prevent partial updates in the event of failures.

Security

All administrative operations shall be protected using strong authentication, role-based authorization, encrypted communication, secure password handling, audit logging, and protection against common web vulnerabilities.

Maintainability

The backend shall be modular, well-documented, and structured to simplify future enhancements, debugging, testing, and maintenance activities.

Interoperability

Backend services shall expose standardized RESTful APIs using JSON payloads, enabling seamless communication with frontend applications and external systems.

2.10 Chapter Summary

This chapter has presented the overall architecture, responsibilities, operational scope, and technical context of the Platform Owner module within the EduManage School Management System. It establishes the architectural foundation upon which the detailed functional requirements are built.

The following chapter provides comprehensive functional specifications for each Platform Owner page, translating approved frontend designs into backend implementation requirements, including business logic, API specifications, database interactions, validation rules, security controls, and acceptance criteria.

CHAPTER THREE

FUNCTIONAL PAGE SPECIFICATIONS

3.1 Introduction

This chapter presents the detailed functional specifications for the Platform Owner (Super Admin) module of the EduManage School Management System. The specifications contained in this chapter are derived from the approved frontend prototype and have been transformed into comprehensive backend requirements intended to guide the design, implementation, testing, and maintenance of the Platform Owner services.

While the frontend prototype defines how users interact with the system through visual components, navigation structures, and user interface controls, it does not define how the underlying system processes requests, validates data, enforces business rules, stores information, or communicates with other platform services. The purpose of this chapter is therefore to bridge that gap by translating the approved user interface into a complete backend specification that software developers can implement with consistency and confidence.

Each page specification describes the expected system behaviour, business objectives, functional requirements, data processing requirements, validation rules, security controls, API endpoints, database interactions, error handling mechanisms, and acceptance criteria necessary for successful backend implementation. Collectively, these specifications define the operational responsibilities of the Platform Owner module and establish a standardized approach for implementing its services.

The Platform Owner module represents the highest level of authority within the EduManage platform and operates across all registered schools. Unlike school-level administrative modules, which manage the operations of a single institution, the Platform Owner module provides centralized management of the entire multi-tenant platform. Consequently, the backend services supporting this module must be designed with particular emphasis on scalability, reliability, security, maintainability, and auditability.

The specifications contained in this chapter are organized according to the major functional pages available within the Platform Owner interface. Each page has been analysed independently to identify its business purpose, user interactions, required backend processes, system integrations, database requirements, validation constraints, security considerations, and expected application behaviour. This modular approach allows backend developers to implement each functional area independently while maintaining consistency across the overall architecture of the system.

To promote standardization throughout the Platform Owner module, every page specification follows a common documentation structure. Although the functionality differs from one page to another, each specification includes the same core sections to ensure completeness and maintain uniform documentation standards. These sections include:

- Introduction
- Business Objective
- Page Overview
- Actors and User Roles
- Functional Requirements
- User Interface Components
- Business Rules
- Backend API Specification
- Database Requirements
- Validation Rules
- Security and Permissions
- Error Handling

- Acceptance Criteria
- Backend Implementation Notes
- Data Flow

Maintaining this standardized structure ensures that each functional page is documented consistently, making the specification easier to understand, implement, review, and maintain throughout the software development lifecycle.

The backend APIs proposed throughout this chapter follow RESTful architectural principles and are intended to provide secure, modular, and scalable communication between the frontend application and backend services. Each endpoint has been designed to support the business processes identified during the frontend analysis while allowing future enhancements without requiring significant architectural modifications.

Similarly, the database recommendations provided for each page identify the primary entities, relationships, and storage requirements inferred from the frontend functionality. These recommendations are intended to guide database designers in developing a normalized, efficient, and scalable relational database capable of supporting current requirements while accommodating future platform growth.

Security has been considered a fundamental requirement throughout every specification. Since the Platform Owner possesses unrestricted administrative privileges across the EduManage platform, every backend operation described in this chapter assumes strong authentication, Role-Based Access Control (RBAC), comprehensive audit logging, secure communication over HTTPS, rigorous input validation, and appropriate protection against unauthorized access and malicious activity.

It is important to note that this chapter focuses exclusively on the backend functional behaviour required to support the approved frontend design. Visual presentation, styling, responsive layouts, and frontend implementation details are outside the scope of this specification except where they directly influence backend processing or system behaviour.

The functional specifications presented in the following sections serve as the primary technical reference for backend developers responsible for implementing the Platform Owner module. They also provide a common understanding among software architects, database designers, quality assurance engineers, project managers, and other stakeholders involved in the development of the EduManage School Management System.

The remainder of this chapter presents the detailed specifications for each Platform Owner page, beginning with the Platform Overview Dashboard and continuing through Schools Management, Subscriptions and Billing, Analytics and Reports, Configuration, Platform Administrator Management, Support and Helpdesk, System Monitoring, Developer Tools, and Emergency Controls. Together, these specifications define the complete functional behaviour expected from the Platform Owner backend module.

3.2 Platform Overview (SAHome.jsx)

Platform Owner Specification Document

Module

Super Admin

Page

SAHome.jsx

3.2.1 Introduction

The Platform Overview page serves as the landing dashboard for the Platform Owner after successful authentication. It provides a consolidated view of platform health, operational metrics, financial performance, school growth, recent platform activities, and communication tools.

This page acts as the primary decision-support interface for monitoring the overall performance of the School Management System.

3.3.2 Business Objective

The Platform Overview enables the Platform Owner to:

- Monitor platform-wide performance.
- Track school growth and registrations.
- Monitor financial performance.
- Observe recent platform activities.
- Monitor school approval status.
- View high-level educational statistics.
- Communicate with schools directly from the dashboard.

The dashboard is intended to support strategic decision-making rather than day-to-day school operations.

3.2.3 Page Overview

The dashboard is composed of the following sections:

A. Dashboard Header

Displays:

- Page Title
- Welcome Message

B. Primary Statistics

Displays platform KPIs:

- Total Schools
- Total Students
- Monthly Revenue
- Active Schools

Each card includes:

- Metric value
- Trend
- Comparison with previous period

These are presented using reusable `StatCard` components.

C. Secondary Statistics

Displays additional operational metrics:

- Primary Schools
- Secondary Schools
- Total Teachers
- Total Parents
- Pending Applications
- Overdue Payments

D. Revenue Growth

Displays historical revenue growth and subscriptions using a line chart.

E. School Distribution

Displays schools grouped by district using a doughnut chart.

F. Monthly School Registrations

Displays new school registrations using a bar chart.

G. Recent Activity

Displays recent platform events, including:

- School approvals
- Payment notifications
- Teacher registrations
- School applications
- Financial events

H. Send Message Modal

Allows the Platform Owner to send notifications to:

- A specific school
- All schools (broadcast)

The modal includes:

- School selector
- Message type
- Message body

I. Success Notification

Displays confirmation after a message has been sent.

3.2.4 Actors / User Roles

Role	Permission
Platform Owner	Full Access
School Administrator	No Access
Teacher	No Access
Student	No Access

Authentication:

- JWT Required

Authorization:

`dashboard.view`

3.2.5 Functional Requirements

3.2.5.1 Dashboard Statistics

Purpose

Provide a high-level overview of platform performance.

Expected Behaviour

The system shall display:

- Total Schools
- Total Students
- Monthly Revenue
- Active Schools

These values should be calculated dynamically by the backend rather than being hard-coded.

3.2.5.2 Educational Statistics

Purpose

Provide educational distribution metrics.

The system shall display:

- Number of Primary Schools
- Number of Secondary Schools
- Total Teachers
- Total Parents
- Pending Applications
- Overdue Payments

3.2.5.3 Revenue Growth

Purpose

Visualize platform financial performance.

Expected Behaviour

Display monthly revenue and subscription trends.

The backend shall return time-series data suitable for chart rendering.

3.2.5.4 Schools by District

Purpose

Display geographical distribution of registered schools.

The backend shall aggregate schools by district.

3.2.5.5 Monthly School Registrations

Purpose

Display school growth over time.

The backend shall provide monthly registration counts grouped by month.

3.2.5.6 Recent Activity

Purpose

Provide real-time operational awareness.

The activity feed shall display events such as:

- School approval
- New school registration
- Teacher registration
- Subscription payment
- Payment issues

Each activity should include:

- Activity type
- Description
- Timestamp

3.2.5.7 Send Message

Purpose

Allow the Platform Owner to communicate with schools.

The Platform Owner shall:

- Select a school
- Select all schools
- Choose message priority (Info, Warning, Urgent)
- Compose a message
- Send the notification

The frontend currently supports three message priorities and allows broadcasting to all schools.

3.2.5.8 Toast Notification

After a successful message submission, the system shall display a confirmation notification to the sender.

3.2.6 User Interface Components

Component	Description
Dashboard Header	Displays page title and welcome message
Statistic Cards	Show key platform metrics
Revenue Chart	Visualizes revenue and subscriptions
District Chart	Displays schools by district
Registration Chart	Shows monthly school registrations
Recent Activity Feed	Lists recent platform events
Send Message Modal	Sends notifications to schools
Toast Notification	Confirms successful message delivery

3.2.7 Business Rules

- Dashboard data shall reflect the latest available platform information.
- Statistics shall be generated dynamically.
- Recent activity should be ordered from newest to oldest.
- Broadcast messages shall be delivered to all active schools.
- School-specific messages shall only be delivered to the selected school.
- A message cannot be sent unless a recipient and message body are provided.
- Message priority shall influence how notifications are presented to recipients.

3.2.8 Backend API Specification

Dashboard Summary

Endpoint

GET /api/superadmin/dashboard/summary

Response

```
{
  "totalSchools":247,
  "totalStudents":68430,
  "monthlyRevenue":112000000,
  "activeSchools":231,
  "primarySchools":164,
  "secondarySchools":83,
  "teachers":4218,
  "parents":12640,
  "pendingApplications":16,
  "overduePayments":23
}
```

Revenue Statistics

GET /api/superadmin/dashboard/revenue

District Distribution

GET /api/superadmin/dashboard/districts

Monthly Registrations

GET /api/superadmin/dashboard/school-growth

Recent Activity

GET /api/superadmin/dashboard/activity

Send Notification

POST /api/superadmin/dashboard/messages

Request

```
{
  "recipient": "all",
  "priority": "warning",
  "message": "System maintenance scheduled for tonight."
}
```

Response

```
{
  "success": true,
  "message": "Notification sent successfully."
}
```

3.2.9 Database Requirements

The dashboard retrieves data from multiple tables rather than storing its own records.

Likely data sources include:

Table	Purpose
schools	School statistics and registrations
students	Student counts
teachers	Teacher counts
parents	Parent counts
subscriptions	Revenue and subscription metrics
payments	Revenue and overdue payments
activity_logs	Recent platform events
notifications	Sent messages and broadcasts

3.2.10 Validation Rules

Message Validation

- School selection is required unless sending to all schools.

- Message body is required.
- Message body should not exceed the configured maximum length.
- Priority must be one of: `info`, `warning`, or `urgent`.

Dashboard Validation

- Numeric values must not be negative.
- Empty datasets should return valid empty collections rather than causing UI failures.

3.2.11 Security & Permissions

Authentication

- JWT Authentication Required

Authorization

`dashboard.view`
`dashboard.notify`

Security Measures

- Only Platform Owners may access dashboard data.
- Notification requests must be authenticated and authorized.
- All notification actions should be recorded in an audit log.
- Broadcast messaging should be rate-limited to prevent accidental or abusive mass notifications.

3.2.12 Error Handling

Error	HTTP Status
Invalid JWT	401 Unauthorized
Permission Denied	403 Forbidden
Invalid Message	400 Bad Request
School Not Found	404 Not Found
Internal Server Error	500 Internal Server Error

The frontend should display clear error messages while avoiding disclosure of internal system details.

3.2.13 Acceptance Criteria

The dashboard is considered complete when:

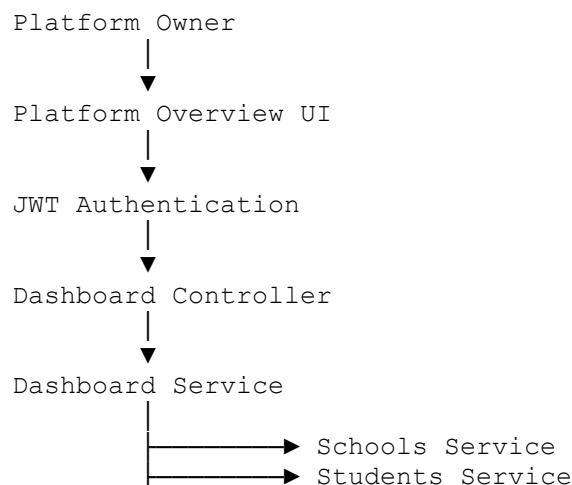
- Platform statistics load successfully.
- Charts render correctly with backend data.
- Recent activity is displayed in chronological order.
- District distribution reflects current school data.
- Revenue trends are calculated accurately.
- Messages can be sent to individual schools.
- Broadcast messages can be sent to all schools.
- Success notifications appear after successful message delivery.
- Unauthorized users cannot access the dashboard.

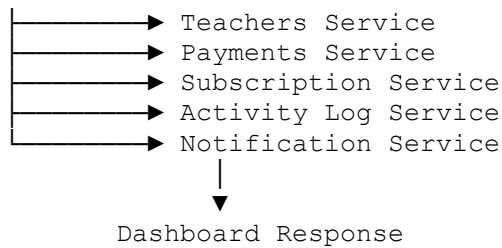
3.2.14 Backend Implementation Notes

The Platform Overview is an **aggregation layer** rather than a standalone business module. The backend should avoid querying multiple tables independently for every request. Instead, implement a dedicated **Dashboard Service** responsible for collecting, summarizing, and caching data from schools, students, subscriptions, payments, and activity logs.

Since this page is expected to be accessed frequently after login, caching expensive calculations (for example, total counts and monthly trends) can improve performance while still allowing near-real-time updates.

3.2.15 Data Flow





Backend observation

The frontend currently defines static datasets for revenue trends, district distribution, school growth, recent activity, and school lists directly in the component. As the backend is implemented, these should be replaced with API responses. A dedicated dashboard aggregation service will keep the frontend simple while avoiding multiple independent API calls for every dashboard widget.

3.3 Schools Management (SASchools.jsx)

Platform Owner Specification Document

Module

Super Admin

Page

SASchools.jsx

3.3.1 Introduction

The Schools Management page enables the Platform Owner to manage all schools registered on the EduManage platform throughout their entire lifecycle. It supports registration, approval, rejection, suspension, deletion, billing, document verification, school administration, and profile maintenance.

This module serves as the central management interface for all tenant institutions using the platform.

3.3.2 Business Objective

The objective of this module is to enable the Platform Owner to:

- Register new schools.
- View registered schools.
- Approve or reject school applications.
- Suspend or reactivate schools.
- Delete schools when necessary.
- View detailed school profiles.
- Manage subscription billing.
- Review uploaded legal documents.
- Update school registration information.
- Impersonate School Administrators for support purposes.

3.3.3 Page Overview

The page is organized into several functional areas.

A. Header

Displays:

- Schools Management title
- Page description
- **Add School** action

B. Search & Filtering

Allows filtering by:

Status

- All
- Active
- Pending
- Suspended

School Level

- All Levels
- Primary
- Secondary

Search

Search by:

- School Name
- Location

C. Schools Table

Each school record displays:

- School Name
- Level
- Location
- Student Count
- Monthly Bill
- Contact Person
- Subdomain
- Status
- Payment Status
- Available Actions

The monthly bill is currently calculated in the frontend as:

Student Count × UGX 2,000 per student per month.

D. School Actions

Available actions include:

- View School
- Approve School
- Reject School
- Suspend School
- Login as School Admin
- Delete School

The available actions depend on the school's current status.

E. Detailed School Profile

The View modal displays:

- School Information
- Billing Breakdown
- Contact Information
- Registration Documents
- School Name Update
- Legal Document Upload

F. School Registration

Allows Platform Owner to register schools manually.

Sections include:

- School Information
- Contact Information
- Portal Information
- Billing Preview
- License Upload

G. Document Viewer

Allows viewing and downloading uploaded Ministry of Education (MoES) license documents.

3.3.4 Actors / User Roles

Role	Permission
Platform Owner	Full Access
School Administrator	Read Own School Only
Teacher	No Access
Student	No Access

Authentication

- JWT Required

Authorization

`schools.manage`

3.3.5 Functional Requirements

3.3.5.1 Register School

Purpose

Create a new school on the platform.

Required Information

School

- School Name
- School Level
- District
- Physical Address
- Number of Students

Contact

- Contact Person
- Phone Number
- Email

Portal

- Subdomain

Documents

- MoES License

The frontend also provides a live billing preview based on the entered student count.

3.3.5.2 View School

Display complete school profile including:

- General Information
- Contact Details
- Billing
- Documents
- Payment Status
- School Status

3.3.5.3 Approve School

Approving a school shall:

- Activate school account.
- Activate subdomain.
- Send welcome email.
- Generate administrator account (if not already created).

- Start billing cycle.

These expected outcomes are explicitly communicated in the approval modal.

3.3.5.4 Reject School

The Platform Owner shall provide:

- Rejection Reason

The rejection reason shall be emailed to the applicant.

3.3.5.5 Suspend School

Suspending a school shall:

- Disable access for users associated with the school.
- Preserve existing data.
- Stop new logins until reactivated.

3.3.5.6 Delete School

Deleting a school shall permanently remove:

- Students
- Teachers
- Attendance
- Grades
- Payments
- School configuration
- Other dependent records

The frontend warns that this operation is irreversible and requests the administrator's password.

3.3.5.7 Login as School Administrator

The Platform Owner shall be able to impersonate the School Administrator.

Business Rules

- Session shall be audited.
- Original administrator credentials shall never be exposed.
- Platform Owner shall be able to exit impersonation safely.

3.3.5.8 Update School Name

Allows updating the registered school name.

Requirements:

- New Name
- Supporting Legal Document

Name changes should only take effect after verification.

3.3.5.9 Document Management

The Platform Owner shall:

- View uploaded MoES license.
- Download document.
- Verify authenticity.
- Reject invalid documentation.

3.3.5.10 Billing Preview

The monthly subscription amount shall be calculated using:

`Monthly Bill = Student Count × Platform Rate`

The platform rate should be configurable through the Configuration module rather than hard-coded.

3.3.6 User Interface Components

Component	Description
Search Box	Search schools by name or location
Status Filter	Filter by lifecycle status
Level Filter	Filter by school level
Schools Table	Displays registered schools
Billing Preview	Calculates monthly subscription
School Details Modal	Displays complete profile
Registration Modal	Registers new schools
Approval Modal	Confirms approval

Component	Description
Rejection Modal	Captures rejection reason
Suspension Modal	Confirms suspension
Delete Modal	Confirms permanent deletion
Document Viewer	Displays uploaded license

3.3.7 Business Rules

- School subdomains shall be unique.
- Email addresses shall be unique.
- Schools cannot be approved without required documentation.
- Deleted schools cannot be recovered through the application.
- Billing shall be based on the configured pricing policy.
- Every administrative action shall be recorded in audit logs.
- School impersonation shall always be logged.
- Uploaded licenses must be verified before activation.

3.3.8 Backend API Specification

Register School

POST /api/superadmin/schools

List Schools

GET /api/superadmin/schools

Supports:

- Search
- Status filter
- Level filter
- Pagination

School Details

GET /api/superadmin/schools/{id}

Approve School

POST /api/superadmin/schools/{id}/approve

Reject School

POST /api/superadmin/schools/{id}/reject

Request

```
{  
  "reason": "Incomplete registration documents."  
}
```

Suspend School

POST /api/superadmin/schools/{id}/suspend

Delete School

DELETE /api/superadmin/schools/{id}

Update School Name

PUT /api/superadmin/schools/{id}/name

Upload Legal Document

POST /api/superadmin/schools/{id}/documents

Download Document

GET /api/superadmin/schools/{id}/documents/{documentId}

Impersonate School Administrator

POST /api/superadmin/schools/{id}/impersonate

3.3.9 Database Requirements

Core tables include:

- schools
- school_contacts
- school_documents
- subscriptions
- subscription_pricing
- school_billing
- school_status_history
- audit_logs

Relationships:

- One School → Many Students
- One School → Many Teachers
- One School → Many Documents
- One School → One Subscription
- One School → Many Billing Records

3.3.10 Validation Rules

Registration

Required:

- School Name
- School Level
- District
- Address
- Contact Person
- Phone
- Email
- Subdomain

Validation:

- Valid email
- Valid phone format
- Positive student count
- Unique subdomain
- Supported document type (PDF/JPG/PNG)
- File size within configured limit

Approval

- School must not already be active.
- Required documentation must exist.
- Mandatory fields must be complete.

Deletion

- Administrator password required.
- Optional secondary confirmation (recommended MFA).

3.3.11 Security & Permissions

Authentication

- JWT Authentication Required

Authorization

```
schools.manage
schools.impersonate
schools.delete
schools.documents
```

Security Measures

- Record all administrative actions.
- Restrict impersonation to Platform Owners.
- Encrypt sensitive contact information where appropriate.
- Validate uploaded files and scan for malware.
- Protect destructive actions with re-authentication.

3.3.12 Error Handling

Error	HTTP Status
Invalid JWT	401 Unauthorized
Permission Denied	403 Forbidden
Duplicate Subdomain	409 Conflict
Duplicate Email	409 Conflict
Invalid Document	400 Bad Request
School Not Found	404 Not Found
Validation Error	400 Bad Request
Internal Server Error	500 Internal Server Error

The frontend should preserve entered form data after validation failures and display user-friendly messages.

3.3.13 Acceptance Criteria

The module is considered complete when:

- Schools can be registered successfully.
- Schools can be searched and filtered.
- School details are displayed accurately.
- Pending schools can be approved or rejected.
- Suspended schools lose platform access.
- Billing is calculated correctly.
- Legal documents can be viewed and downloaded.
- School names can be updated with supporting documentation.
- Platform Owner can impersonate School Administrators.
- Unauthorized users cannot perform administrative actions.

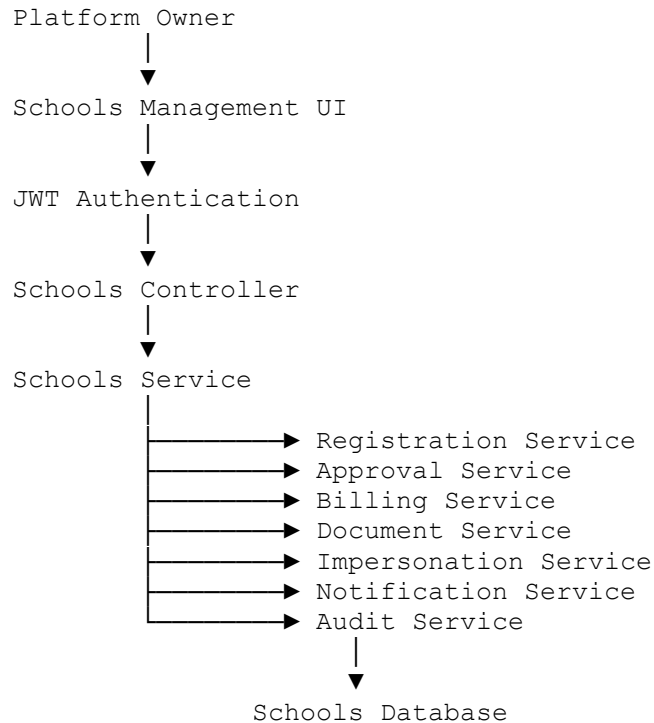
3.3.14 Backend Implementation Notes

This is the **central tenant management module** of the platform. Rather than implementing all logic inside a single controller, the backend should separate responsibilities into services such as:

- **School Service** (registration and profile management)
- **Approval Service** (approval and rejection workflows)
- **Billing Service** (subscription and monthly charges)
- **Document Service** (upload, storage, and verification)
- **Impersonation Service** (secure login as School Administrator)
- **Audit Service** (logging all administrative actions)

One important observation is that the frontend currently calculates the monthly bill using a constant (`PRICE_PER_STUDENT = 2000`). In production, this pricing should come from the platform configuration or subscription plans so that changes do not require frontend code updates.

3.3.15 Data Flow



NOTE

This is the backbone of the Platform Owner module. It defines the complete lifecycle of a tenant on the platform. It will become the reference for several backend modules (schools, billing, subscriptions, documents, notifications, and auditing),.

3.4 Subscriptions & Billing (SASubscription.jsx)

Platform Owner Specification Document

Module

Super Admin

Page

3.4.1 Introduction

The Subscriptions & Billing page enables the Platform Owner to configure the platform's subscription pricing model, monitor subscription performance, calculate recurring revenue, track overdue payments, and manage billing communications with schools.

This module provides centralized control over the financial operations of the School Management System.

3.4.2 Business Objective

The objective of this module is to enable the Platform Owner to:

- Configure the global subscription price per student.
- Monitor monthly recurring revenue.
- View subscription status of all schools.
- Identify overdue accounts.
- Send payment reminders.
- Forecast revenue based on pricing changes.
- Support financial planning and subscription management.

3.4.3 Page Overview

The page consists of five major sections.

A. Header

Displays:

- Subscriptions & Billing title
- Current price per student
- **Set Price Per Student** button

B. Financial Summary

Displays:

- Active Students
- Monthly Revenue
- Total Schools
- Overdue Schools

These summary cards are calculated from the school dataset and current pricing configuration.

C. Subscription Table

Displays all schools with:

- School Name
- School Level
- Student Count
- Monthly Bill
- Due Date
- Subscription Status
- Available Actions

D. Overdue Accounts

Displays schools with overdue subscription payments.

Available actions:

- Send Reminder
- Suspend School

E. Pricing Configuration

Allows the Platform Owner to:

- Configure platform-wide price per student.
- Preview projected revenue.
- Save updated pricing.

The frontend enforces a minimum and maximum price and provides a live revenue preview.

F. Payment Reminder

Allows the Platform Owner to send reminders through:

- Email
- SMS
- In-App Notification

3.4.4 Actors / User Roles

Role	Permission
Platform Owner	Full Access
Finance Administrator (<i>optional</i>)	Configurable
School Administrator	View Own Subscription Only
Teacher	No Access
Student	No Access

Authentication

- JWT Required

Authorization

`subscriptions.manage`

3.4.5 Functional Requirements

3.4.5.1 View Subscription Dashboard

Purpose

Provide an overview of subscription performance.

The dashboard shall display:

- Active Students
- Monthly Revenue
- Total Schools
- Overdue Schools

Values shall be calculated dynamically by the backend.

3.4.5.2 View School Subscriptions

Purpose

Display subscription details for every registered school.

Each record shall include:

- School Name

- School Level
- Student Count
- Monthly Bill
- Due Date
- Subscription Status

3.4.5.3 Configure Price Per Student

Purpose

Allow the Platform Owner to configure the platform-wide subscription price.

Expected Behaviour

The system shall:

- Validate the entered price.
- Save the new pricing policy.
- Recalculate all monthly subscription amounts.
- Update revenue projections.

The current frontend limits the value to a configurable range and rounds to the nearest permitted increment before saving.

3.4.5.4 Revenue Forecast

Purpose

Provide an estimate of expected monthly revenue.

The system shall calculate:

$$\text{Revenue} = \text{Total Active Students} \times \text{Price Per Student}$$

The pricing modal also previews revenue for schools of different sizes and for the platform as a whole.

3.4.5.5 View Overdue Accounts

Purpose

Identify schools with unpaid subscriptions.

Each overdue record shall include:

- School Name
- Student Count
- Outstanding Monthly Amount
- Due Date

3.4.5.6 Send Payment Reminder

The Platform Owner shall:

- Select communication channels.
- Customize reminder message.
- Send reminder.

Supported channels:

- Email
- SMS
- In-App Notification

3.4.5.7 Suspend Overdue School

The Platform Owner may suspend schools that fail to pay after the configured grace period.

Suspension shall follow the platform's billing policy.

3.4.6 User Interface Components

Component	Description
Summary Cards	Display subscription KPIs
Subscription Table	Displays school billing information
Overdue Tab	Lists overdue subscriptions
Pricing Modal	Configures billing policy
Revenue Preview	Displays projected revenue
Reminder Modal	Sends payment reminders
Status Badge	Displays subscription status

3.4.7 Business Rules

- Billing is based on the number of active students.

- One pricing policy applies platform-wide.
- Student count changes affect future billing calculations.
- Schools with overdue payments remain visible until payment is settled.
- Payment reminders may be delivered through multiple channels simultaneously.
- Revenue calculations use only active schools unless otherwise specified.
- Pricing changes should not retroactively modify historical invoices.

3.4.8 Backend API Specification

Subscription Dashboard

GET /api/superadmin/subscriptions/summary

Response

```
{
  "activeStudents":3440,
  "monthlyRevenue":6880000,
  "totalSchools":7,
  "overdueSchools":2
}
```

List School Subscriptions

GET /api/superadmin/subscriptions

Supports:

- Pagination
- Search
- Status filtering

Update Pricing

PUT /api/superadmin/subscriptions/pricing

Request

```
{
  "pricePerStudent":2500
}
```

Response

```
{
```

```
"success":true,
"pricePerStudent":2500
}
```

Revenue Forecast

GET /api/superadmin/subscriptions/revenue-preview

Send Payment Reminder

POST /api/superadmin/subscriptions/reminders

Request

```
{
  "schoolId":3,
  "channels":["email","sms","in-app"],
  "message":"Your subscription payment is overdue."
}
```

Suspend Subscription

POST /api/superadmin/subscriptions/{schoolId}/suspend

3.4.9 Database Requirements

Recommended tables:

subscriptions

- id
- school_id
- status
- current_price
- billing_cycle
- next_due_date

subscription_pricing

- id
- price_per_student
- effective_date

- created_by

invoices

- id
- subscription_id
- student_count
- amount
- issue_date
- due_date
- payment_status

payments

- id
- invoice_id
- amount
- payment_date
- payment_method

reminder_logs

- id
- school_id
- channel
- sent_by
- sent_at

Relationships:

- One School → One Active Subscription
- One Subscription → Many Invoices
- One Invoice → Many Payments (if partial payments are supported)

3.4.10 Validation Rules

Pricing

- Price is required.
- Price must fall within the configured minimum and maximum values.
- Price should follow the allowed increment (for example, 500 UGX).

Subscription

- Student count must not be negative.
- Due date must be valid.
- Subscription status must be one of: `active`, `pending`, or `overdue`.

Payment Reminder

- At least one delivery channel must be selected.
- Reminder message is required.
- Selected school must exist.

3.4.11 Security & Permissions

Authentication

- JWT Authentication Required

Authorization

```
subscriptions.manage
subscriptions.pricing
subscriptions.reminders
```

Security Measures

- Record all pricing changes in an audit log.
- Require elevated privileges for pricing modifications.
- Protect billing APIs with authorization checks.
- Ensure reminder messages are logged for compliance.

3.4.12 Error Handling

Error	HTTP Status
Invalid JWT	401 Unauthorized
Permission Denied	403 Forbidden
Invalid Pricing	400 Bad Request
School Not Found	404 Not Found
Reminder Delivery Failed	500 Internal Server Error
Internal Server Error	500 Internal Server Error

The frontend should preserve unsaved pricing values when validation fails and display clear feedback to the user.

3.4.13 Acceptance Criteria

The module is considered complete when:

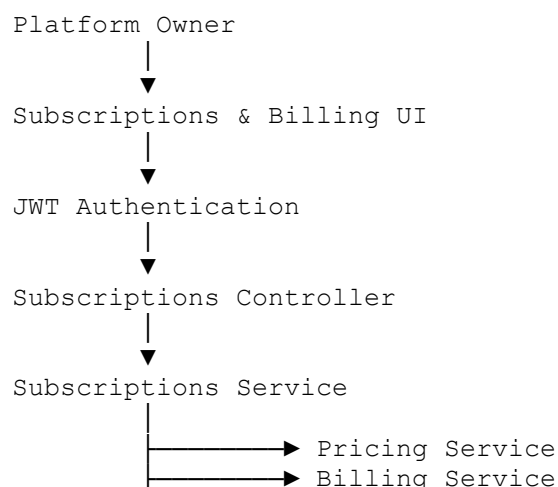
- Subscription summary statistics load successfully.
- School subscription data is displayed accurately.
- Pricing can be updated within the allowed limits.
- Revenue preview reflects pricing changes.
- Overdue schools are listed correctly.
- Payment reminders can be sent through selected channels.
- Suspension actions are available for overdue schools.
- Unauthorized users cannot modify subscription data.

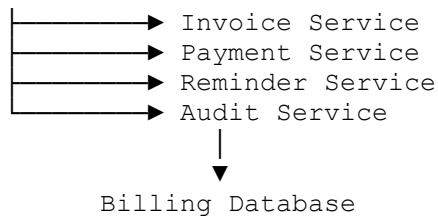
3.4.14 Backend Implementation Notes

The Subscriptions & Billing module should be implemented as a dedicated billing service rather than embedding financial calculations within the Schools module. Pricing policies, invoices, payments, and reminders should be separated into distinct services to support future enhancements such as multiple subscription plans, promotional pricing, or different billing models.

A key observation from the frontend is that the price per student is configurable rather than fixed. The backend should therefore persist pricing history with effective dates so that historical invoices remain accurate even after the current pricing changes.

3.4.15 Data Flow





NOTE

This module forms the **financial engine** of the platform. Together with **Schools Management**, it defines the commercial side of the SaaS application: schools are onboarded through the Schools module, while this page governs how those schools are billed and monitored over time. The frontend also hints at future scalability by separating pricing configuration from invoice management, which is a strong architectural direction for the backend.

3.5 Analytics & Reports (SAAalytics.jsx)

Platform Owner Specification Document

Module: Super Admin

Page: SAAalytics.jsx

3.5.1 Page Information

Item	Description
Module	Super Admin
Page	Analytics & Reports
Route	<code>/superadmin/analytics</code> (assumed from frontend routing)
Authorized Role	Platform Owner / Super Administrator
Authentication	JWT Required
Authorization	Role = Super Admin

3.5.2 Purpose

The Analytics & Reports page provides the Platform Owner with a centralized view of platform performance through dashboards, charts, business intelligence metrics, revenue forecasting, school growth statistics, retention trends, district distribution, subscription opportunities, and report generation.

The page enables strategic monitoring of the entire School Management System rather than operational management of a single school.

3.5.3 Functional Requirements

The system shall allow the Platform Owner to:

3.5.3.1 View Platform Metrics

Display:

- Total Schools
- Total Students
- Monthly Revenue
- Churn Rate

These appear as the first row of statistic cards in the UI.

3.5.3.2 View Business Intelligence Metrics

Display

- Monthly Recurring Revenue (MRR)
- Annual Recurring Revenue Projection (ARR)
- Lifetime Value (LTV)
- Customer Acquisition Cost (CAC)

These are shown as a second row of statistic cards.

3.5.3.3 Platform Growth

Display a line chart showing monthly growth of:

- Schools
- Students

The backend should return historical data grouped by month.

3.5.3.4 Monthly Revenue

Display monthly revenue using bar charts.

The frontend expects:

- Actual revenue
- Forecast revenue

The backend must provide both datasets.

3.5.3.5 Revenue Forecast

Display projected revenue for future months.

Forecast values may be generated using business rules or forecasting algorithms.

3.5.3.6 Retention Rate

Display historical customer retention percentages.

3.5.3.7 Schools by District

Display school distribution using a doughnut chart.

The backend should aggregate schools by district.

3.5.3.8 Upsell Opportunities

The frontend displays schools approaching their subscription limits.

Each record contains:

- School name
- Current subscription
- Recommended subscription
- Current students
- Maximum allowed students
- Percentage utilization

This means the backend must calculate utilization percentage instead of relying on the frontend.

3.5.3.9 Report Generator

Allow the Platform Owner to:

- Select report type
- Select start date
- Select end date
- Choose output format (PDF / Excel)
- Generate report
- Download previous report

These controls are defined at the bottom of the component.

3.5.4 UI Components Identified

From the React component, the page contains:

- Date Range Filter
- Statistics Cards
- Line Charts
- Bar Charts
- Doughnut Chart
- Upsell Cards
- Report Generator
- Download Button

3.5.5 Backend API Requirements

Instead of one large endpoint, I recommend exposing focused endpoints.

Dashboard Summary

GET /api/superadmin/analytics/summary

Returns

```
{
  "totalSchools":247,
  "totalStudents":68430,
  "monthlyRevenue":112000000,
  "churnRate":3.2,
  "mrr":112000000,
  "arr":1340000000,
  "ltv":2800000,
  "cac":180000
}
```

Platform Growth

GET /api/superadmin/analytics/platform-growth

Returns

```
[
  {
    "month":"Aug",
    "schools":180,
    "students":41000
  }
]
```

Revenue

GET /api/superadmin/analytics/revenue

Returns

Actual

Forecast

Historical

Retention

GET /api/superadmin/analytics/retention

District Distribution

GET /api/superadmin/analytics/districts

Upsell Opportunities

GET /api/superadmin/analytics/upsells

Returns

```
[
  {
    "schoolId":1,
    "school":"Greenhill Academy",
    "currentPlan":"Pro",
    "recommendedPlan":"Enterprise",
    "students":1240,
    "limit":2000,
    "utilization":62
  }
]
```

Generate Report

POST /api/superadmin/reports/generate

Request

```
{
  "reportType":"Revenue Summary",
  "from":"2026-01-01",
  "to":"2026-02-22",
  "format":"PDF"
}
```

Returns

```
{
  "downloadUrl":"..."
}
```

3.5.6 Database Tables Required

Based on the UI, the backend will likely need data from:

- Schools
- Students

- Subscriptions
- Payments
- Platform Users
- Support Tickets
- Districts (or district field within Schools)
- Reports (if generated reports are stored)

3.5.7 Validation Rules

- End date must not be earlier than start date.
- Report type is required.
- Export format is required.
- Only Platform Owner users may generate platform-wide reports.
- Report date range should be validated against available data.

3.5.8 Security Requirements

JWT Authentication required.

Required permission:

`analytics.view`

Report generation:

`analytics.export`

3.5.9 Error Handling

Examples:

Condition	Response
Invalid JWT	401 Unauthorized
Insufficient permissions	403 Forbidden
Invalid date range	400 Bad Request

Condition	Response
Report generation failure	500 Internal Server Error
No analytics data	200 OK with empty datasets

3.5.10 Acceptance Criteria

The page is considered complete when:

- Dashboard statistics load successfully.
- Charts display data for the selected period.
- District distribution is correctly aggregated.
- Upsell opportunities are calculated accurately.
- Reports can be generated in PDF and Excel.
- Report downloads are successful.
- Unauthorized users cannot access the page.

Note

One thing stood out while reviewing this component: **the frontend currently uses hard-coded mock data** (`growthData`, `retentionData`, `districtData`, `revenueBar`, `forecastData`, and `upsells`) defined directly in the file.

That tells us the backend specification should not simply copy these values. Instead, it should define the **dynamic data contracts** (APIs and response formats) that will replace those mock objects. This is exactly the kind of insight a specification document should provide.

3.6 Configuration (SAConfiguration.jsx)

Platform Owner Specification Document

Module

Super Admin

Page

SAConfiguration.jsx

3.6.1 Overview

The Configuration page enables the Platform Owner to configure platform-wide settings that affect every school and user on the system. this page is **the platform's centralized configuration center**. Unlike the Analytics page, this page is **transactional**—it saves and updates system settings rather than displaying business intelligence.

This page naturally breaks into **five independent backend modules**, each with its own responsibility. Instead of one large endpoint, the backend should expose separate endpoints for each tab.

The page contains five configuration categories:

1. Appearance
2. Branding
3. Feature Toggles
4. Integrations
5. Security

These categories are presented as tabs in the UI.

3.6.2 Business Objective

Provide a centralized administration interface where the Platform Owner can:

- Customize the platform's appearance
- Configure branding
- Enable or disable platform features
- Manage third-party integrations
- Configure security policies

Changes made on this page apply globally across the platform.

3.6.3 Access Control

Role	Access
Platform Owner	Full Access
School Administrator	No Access
Teacher	No Access
Student	No Access

Authentication:

- JWT Required

Authorization:

`configuration.manage`

3.6.4 Functional Requirements

3.6.4.1 Appearance Settings

The Platform Owner shall be able to:

- Enable Dark Mode
- Disable Dark Mode

The frontend currently stores the dark mode preference in browser local storage for demonstration. In production, the backend should persist this preference so it can be applied consistently across sessions and devices.

Backend API

`GET /api/superadmin/configuration/appearance`

`PUT /api/superadmin/configuration/appearance`

Example Response

```
{
  "darkMode":true
}
```

3.6.4.2 Branding Settings

The Platform Owner shall configure:

- Platform Name
- Support Email
- Primary Theme Color
- Platform Logo
- Email Footer

These fields are displayed in the Branding tab.

Validation

Platform Name

- Required
- Maximum 100 characters

Support Email

- Valid email format

Primary Color

- Must be a valid HEX color

Logo

- PNG/JPG/SVG only
- Maximum file size (e.g., 5 MB)

Email Footer

- Maximum 500 characters

Backend API

GET /api/superadmin/configuration/branding

PUT /api/superadmin/configuration/branding

POST /api/superadmin/configuration/logo

3.6.4.3 Feature Toggles

The frontend defines these configurable platform features:

- SMS Notifications
- Email Notifications
- Biometric Attendance
- MTN Mobile Money
- Airtel Money
- AWS S3 Storage
- Two-Factor Authentication
- API Access

The Platform Owner shall be able to:

- Enable a feature
- Disable a feature
- Save feature configuration

Backend API

GET /api/superadmin/configuration/features

PUT /api/superadmin/configuration/features

Example Response

```
[
  {
    "key": "sms",
    "enabled": true
  },
  {
    "key": "email",
    "enabled": true
  }
]
```

3.6.4.4 Integration Settings

The page supports configuration of third-party services.

The frontend includes fields for:

- Africa's Talking API Key
- MTN Mobile Money API Key
- Airtel Money API Key
- SendGrid API Key
- AWS Access Key

- AWS Secret Key
- AWS S3 Bucket Name

Functional Requirements

The system shall:

- Store encrypted API credentials.
- Never expose secret keys in API responses.
- Allow authorized administrators to update credentials.
- Validate required fields before saving.

Backend API

GET /api/superadmin/configuration/integrations

PUT /api/superadmin/configuration/integrations

Security note: Secrets such as API keys should be encrypted at rest, and responses should mask them (for example, showing only the last few characters where appropriate).

3.6.4.5 Security Settings

The frontend allows configuration of:

- Session Timeout
- Maximum Login Attempts
- IP Whitelist
- Enforce Two-Factor Authentication

Validation

Session Timeout

- Minimum 5 minutes
- Maximum 1440 minutes

Maximum Login Attempts

- Minimum 3
- Maximum 10

IP Whitelist

- Validate IPv4/IPv6 formats
- Ignore blank lines

Backend API

GET /api/superadmin/configuration/security

PUT /api/superadmin/configuration/security

3.6.5 Database Design

Rather than storing everything in one large row, I'd recommend a general `platform_configuration` table with a key/value structure for flexibility. Examples of keys include:

- `appearance.dark_mode`
- `branding.platform_name`
- `branding.support_email`
- `features.sms`
- `security.session_timeout`

Sensitive integration secrets should be stored securely (encrypted or managed through a secrets solution), not as plain text.

3.6.6 Error Handling

Condition	Response
Invalid JWT	401 Unauthorized
Insufficient Permission	403 Forbidden
Invalid Email	400 Bad Request
Invalid Color Code	400 Bad Request
Invalid API Key Format	400 Bad Request
Upload Failure	500 Internal Server Error

3.6.7 Acceptance Criteria

The module is considered complete when:

- Appearance settings persist after saving.
- Branding changes are reflected throughout the platform.
- Feature toggles enable or disable functionality correctly.
- Integration credentials are securely stored and masked when retrieved.
- Security policies are validated and enforced.
- Only authorized Platform Owners can modify configuration.

3.7 Platform Administrator Management (SAUsers.jsx)

Platform Owner Specification Document

Module

Super Admin

Page

SAUsers.jsx

3.7.1 Introduction

The Platform Administrator Management page enables the Platform Owner to manage administrative users responsible for operating and maintaining the EduManage platform.

The page allows the Platform Owner to:

- View Platform Administrators
- Create Platform Administrators
- Remove Platform Administrators

The backend should also support Role-Based Access Control (RBAC) to allow different administrative responsibilities across the platform.

3.7.2 Business Objective

The objective of this module is to provide centralized management of platform administrative accounts while ensuring that only authorized personnel can perform sensitive platform operations.

The module supports secure delegation of responsibilities across finance, customer support, content management, system monitoring, and platform administration.

3.7.3. Page Overview

The page consists of four major sections.

A. Platform Administrator List

Displays all registered Platform Administrators.

Each record contains:

- Administrator Name
- Email Address
- Role
- Last Login
- Online Status
- Available Actions

B. Add Platform Administrator

Allows the Platform Owner to register a new administrator.

Required fields include:

- Full Name
- Email Address
- Role

The frontend indicates that a welcome email containing login credentials will be sent automatically after successful creation.

C. Remove Platform Administrator

Allows the Platform Owner to remove an administrator.

Before removal, the system requests:

- Administrator Password

The frontend also warns that this action cannot be undone.

D. Toast Notifications

Displays success or warning notifications after administrative actions.

3.7.4. Actors / User Roles

Role	Permission
Platform Owner	Full Access
Super Administrator	Read Only (<i>Optional</i>)
Finance Administrator	No Access
Support Agent	No Access
Content Manager	No Access

Authentication

- JWT Required

Authorization

`platform_admin.manage`

3.7.5. Functional Requirements

3.7.5.1 View Platform Administrators

Purpose

Display all administrative users.

Each administrator record shall include:

- Full Name
- Email
- Role
- Last Login
- Current Status

Status values include:

- Online
- Offline

3.7.5.2 Create Platform Administrator

Purpose

Register a new administrator.

Required Information

- Full Name
- Email
- Role

Expected Behaviour

The backend shall:

- Validate submitted information.
- Create administrator account.
- Generate temporary credentials.
- Send welcome email.
- Record audit log.

3.7.5.3 Remove Platform Administrator

Purpose

Revoke administrator access.

Expected Behaviour

The backend shall:

- Verify Platform Owner password.
- Remove administrator privileges.
- Immediately invalidate all active sessions.
- Record audit log.

3.7.5.4 Administrator Status

The backend shall maintain administrator status.

Supported values:

- Online
- Offline
- Suspended (*recommended*)
- Disabled (*recommended*)

3.7.5.5 Role Assignment

The Platform Owner shall assign one administrative role.

Current frontend roles include:

- Super Admin
- Finance Admin
- Support Agent
- Content Manager

The backend should support future role expansion.

3.7.5.6 Permission Management (*Recommended Enhancement*)

Although not implemented in the current UI, the backend should support assigning granular permissions to Platform Administrators.

Potential permissions include:

- Schools Management
- Subscriptions & Billing
- Analytics & Reports
- Support & Helpdesk
- Platform Configuration
- System Monitoring
- Developer Tools
- Emergency Controls
- User Management

This recommendation is based on the unused `permissionLabels` structure present in the frontend.

3.7.6. User Interface Components

Component	Description
Administrator Table	Displays platform administrators
Add Admin Button	Opens registration modal
Add Admin Modal	Creates new administrator
Remove Button	Removes administrator
Remove Confirmation Modal	Confirms administrator removal
Toast Notification	Displays operation results

3.7.7. Business Rules

- Email addresses shall be unique.
- Platform Owners cannot remove themselves while logged in.
- Administrator removal immediately revokes platform access.
- Administrator passwords shall never be stored or transmitted in plain text.
- Every administrative action shall be recorded in audit logs.
- Welcome emails shall be generated automatically after successful account creation.
- Roles determine the administrator's platform permissions.

3.7.8. Backend API Specification

List Administrators

GET /api/superadmin/admins

Create Administrator

POST /api/superadmin/admins

Request

```
{
  "name": "John Doe",
  "email": "john@edumanage.ug",
  "role": "Support Agent"
}
```

Response

```
{
  "success":true,
  "message":"Administrator created successfully."
}
```

Remove Administrator

DELETE /api/superadmin/admins/{adminId}

Request

```
{
  "password":"*****"
}
```

Update Administrator Role (*Recommended*)

PUT /api/superadmin/admins/{adminId}/role

Assign Permissions (*Recommended*)

PUT /api/superadmin/admins/{adminId}/permissions

3.7.9. Database Requirements

platform_admins

Field	Type
id	UUID
full_name	VARCHAR
email	VARCHAR
password_hash	VARCHAR
role_id	UUID
status	ENUM
last_login	TIMESTAMP
created_at	TIMESTAMP

roles

Field	Type
id	UUID
role_name	VARCHAR

permissions

Field	Type
id	UUID
permission_name	VARCHAR

role_permissions

Field	Type
role_id	UUID
permission_id	UUID

admin_activity_logs

Field	Type
id	UUID
admin_id	UUID
action	VARCHAR
created_at	TIMESTAMP

3.7.10. Validation Rules

Administrator Creation

Required:

- Full Name
- Email
- Role

Validation:

- Valid email format.
- Unique email.
- Supported administrator role.

Administrator Removal

Required:

- Platform Owner password.

Validation:

- Password must match authenticated Platform Owner.
- Target administrator must exist.

3.7.11. Security & Permissions

Authentication

- JWT Authentication Required

Authorization

```
platform_admin.manage  
platform_admin.create  
platform_admin.delete
```

Security Measures

- Passwords must be hashed using a strong algorithm (e.g., bcrypt or Argon2).
- Require re-authentication before deleting administrators.
- Log all account creation and deletion events.
- Immediately revoke active sessions after account removal.
- Prevent administrators from elevating their own privileges.

3.7.12. Error Handling

Error	HTTP Status
Invalid JWT	401 Unauthorized
Permission Denied	403 Forbidden
Duplicate Email	409 Conflict
Invalid Password	400 Bad Request
Administrator Not Found	404 Not Found
Internal Server Error	500 Internal Server Error

The frontend should retain entered values after validation failures and provide clear error feedback.

3.7.13. Acceptance Criteria

The module is considered complete when:

- Platform administrators can be listed.
- New administrators can be created successfully.
- Welcome emails are sent after account creation.
- Administrators can be removed after password verification.
- Removed administrators immediately lose platform access.
- Unauthorized users cannot manage administrator accounts.
- All administrative actions are recorded in audit logs.

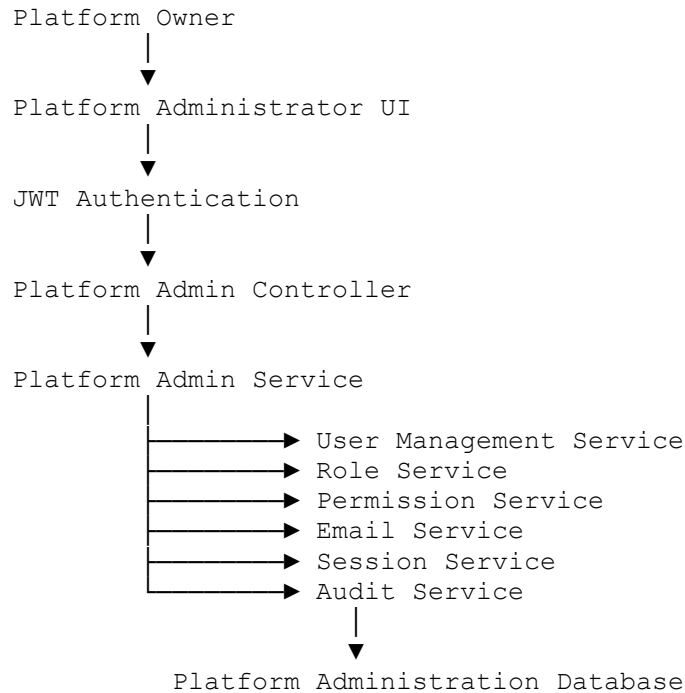
3.7.14. Backend Implementation Notes

This module should be implemented as the **Platform Administration Service**, separate from the Authentication module. While authentication verifies identity, this service manages administrative accounts, roles, and permissions.

The frontend currently exposes only four predefined roles and does not yet provide a permissions interface. However, the presence of the unused `permissionLabels` object strongly suggests that the system is intended to evolve into a full **Role-Based Access Control (RBAC)** implementation. The backend should therefore model roles and permissions separately from the outset to avoid future schema changes.

Additionally, removing an administrator should not permanently delete historical audit records. A **soft delete** or **account deactivation** strategy is preferable to preserve accountability and maintain compliance with auditing requirements.

3.7.15. Data Flow



NOTE

This page complements the **Authentication** module by focusing on **administrative account management** rather than login mechanics. A backend designed with **RBAC from the beginning**—using separate `roles`, `permissions`, and `role_permissions` tables—will align well with the frontend's planned evolution and make it much easier to introduce fine-grained access control as new platform features are added.

3.8 Support & Helpdesk (SASupport.jsx)

Platform Owner Specification Document

Module

Super Admin

Page

3.8.1. Introduction

The **Support & Helpdesk** page provides the Platform Owner with a centralized interface for managing support requests submitted by schools. It enables administrators to monitor ticket status, prioritize issues, respond to schools, and track overall support performance.

This module serves as the primary customer support system for the platform.

3.8.2. Business Objective

The objective of this module is to:

- Receive support requests from schools.
- Track the lifecycle of support tickets.
- Prioritize critical issues.
- Respond to schools efficiently.
- Monitor support performance metrics.
- Improve customer satisfaction through timely issue resolution.

3.8.3. Page Overview

The page consists of six primary sections.

A. Header

Displays:

- Support & Helpdesk title
- Page description

B. Support Summary

Displays key support metrics:

- Open Tickets
- Tickets In Progress
- Resolved Tickets
- Average Response Time

These values should be calculated dynamically by the backend.

C. Ticket Status Filters

Allows filtering tickets by:

- All
- Open
- In Progress
- Resolved

The frontend displays ticket counts alongside each status filter.

D. Ticket Search

Allows searching by:

- Ticket ID
- School Name
- Ticket Subject

The current frontend performs client-side filtering against these fields. In production, search should be handled by the backend to support larger datasets.

E. Tickets Table

Displays:

- Ticket ID
- School Name
- Subject
- Priority
- Status
- Date
- Action

Each ticket includes a **View** action that opens a detailed modal.

F. Ticket Details Modal

Displays:

- School
- Subject
- Priority
- Status
- Date
- Reply editor

The Platform Owner can compose and send a reply to the school.

3.8.4. Actors / User Roles

Role	Permission
Platform Owner	Full Access
Support Agent (<i>future</i>)	Configurable
School Administrator	Create and View Own Tickets
Teacher	Optional (based on platform policy)
Student	No Access

Authentication

- JWT Required

Authorization

`support.manage`

3.8.5. Functional Requirements

3.8.5.1 View Support Dashboard

Purpose

Provide a high-level overview of support operations.

The system shall display:

- Open Tickets
- In Progress Tickets
- Resolved Tickets
- Average Response Time

These metrics should update dynamically.

3.8.5.2 View Support Tickets

Purpose

Display all submitted support tickets.

Each ticket shall include:

- Ticket Number
- School
- Subject
- Priority
- Status
- Date Created

3.8.5.3 Search Tickets

The Platform Owner shall be able to search by:

- Ticket ID
- School Name
- Subject

Search shall support partial and case-insensitive matching.

3.8.5.4 Filter Tickets

The Platform Owner shall filter tickets by:

- Open
- In Progress
- Resolved

The backend should return only matching records.

3.8.5.5 View Ticket Details

The system shall display:

- Ticket information
- School information
- Current status
- Priority
- Date created
- Conversation history (recommended enhancement)

The current frontend displays ticket metadata and a reply field.

3.8.5.6 Reply to Ticket

Purpose

Allow the Platform Owner to communicate with the school.

The Platform Owner shall:

- Enter a reply.
- Submit the response.

The backend shall:

- Save the reply.
- Associate it with the ticket.
- Notify the school through the configured communication channels.

3.8.5.7 Ticket Status Management (*Recommended Enhancement*)

Although not yet implemented in the frontend, the backend should support changing ticket status to:

- Open
- In Progress
- Resolved
- Closed

This allows future UI enhancements without redesigning the backend.

3.8.6. User Interface Components

Component	Description
Summary Cards	Display support metrics
Status Filter Tabs	Filter tickets by workflow status
Search Box	Search support tickets
Tickets Table	Display support requests
Priority Badge	Indicates ticket urgency
Status Badge	Indicates workflow stage
View Button	Opens ticket details
Ticket Modal	Displays ticket details and reply form

Component	Description
Reply Text Area	Allows administrators to respond

3.8.7. Business Rules

- Every ticket shall have a unique ticket number.
- Every ticket belongs to one school.
- Every reply shall be linked to the corresponding ticket.
- Critical tickets should receive higher priority in listings.
- Ticket status changes shall be recorded.
- Schools shall only view their own tickets.
- Platform Owners shall have unrestricted access.
- Empty replies shall not be accepted.

3.8.8. Backend API Specification

Dashboard Summary

GET /api/superadmin/support/summary

Response

```
{
  "open":3,
  "InProgress":2,
  "resolved":3,
  "averageResponseTime":"2.4 hrs"
}
```

List Tickets

GET /api/superadmin/support/tickets

Supports:

- Search
- Status filter
- Pagination

Ticket Details

GET /api/superadmin/support/tickets/{ticketId}

Reply to Ticket

POST /api/superadmin/support/tickets/{ticketId}/reply

Request

```
{
  "message": "Please reset the teacher password and try again."
}
```

Response

```
{
  "success": true,
  "message": "Reply sent successfully."
}
```

Update Ticket Status (*Recommended*)

PATCH /api/superadmin/support/tickets/{ticketId}/status

Request

```
{
  "status": "resolved"
}
```

3.8.9. Database Requirements

Recommended tables:

support_tickets

Field	Description
id	Ticket identifier
school_id	Associated school
subject	Ticket subject
description	Ticket details
priority	Ticket priority
status	Workflow status

Field	Description
created_at	Date created
updated_at	Last updated

support_messages

Field	Description
id	Message identifier
ticket_id	Associated ticket
sender_id	User sending message
message	Reply content
created_at	Timestamp

support_categories (*optional future enhancement*)

Stores ticket categories such as:

- Technical
- Billing
- Training
- Feature Request

3.8.10. Validation Rules

Ticket Reply

- Reply is required.
- Maximum length should follow platform messaging limits.

Ticket Status

Allowed values:

- Open
- In Progress
- Resolved
- Closed

Search

- Ignore letter case.
- Support partial matching.

3.8.11. Security & Permissions

Authentication

- JWT Authentication Required

Authorization

`support.manage`
`support.reply`

Security Measures

- Schools may access only their own tickets.
- All replies shall be recorded in audit logs.
- Validate all user input to prevent injection attacks.
- Protect support endpoints with authorization middleware.

3.8.12. Error Handling

Error	HTTP Status
Invalid JWT	401 Unauthorized
Permission Denied	403 Forbidden
Ticket Not Found	404 Not Found
Invalid Reply	400 Bad Request
Internal Server Error	500 Internal Server Error

The frontend should preserve the drafted reply if submission fails and display a meaningful error message.

3.8.13. Acceptance Criteria

The module is considered complete when:

- Support metrics load successfully.

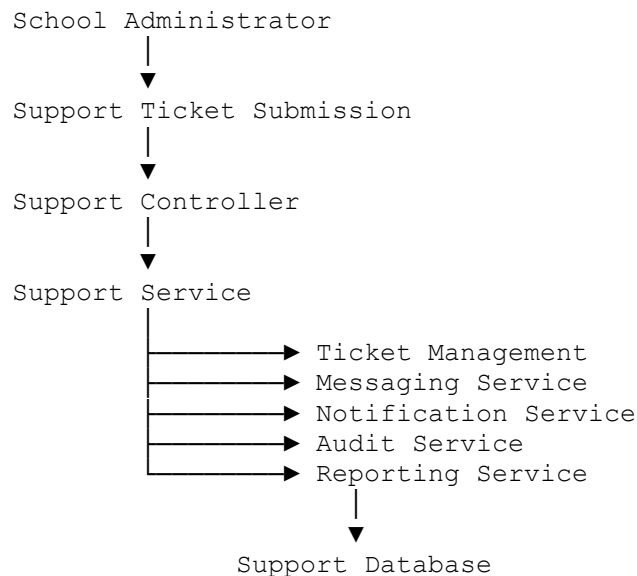
- Tickets can be searched and filtered.
- Ticket details display correctly.
- Replies can be sent successfully.
- Schools receive notifications after replies.
- Ticket status updates are reflected accurately.
- Unauthorized users cannot access support management.

3.8.14. Backend Implementation Notes

The Support module should be implemented as a dedicated ticketing service. Rather than storing only a single reply, the backend should support **conversation threads**, allowing multiple exchanges between schools and Platform Owners. This approach provides a complete communication history and accommodates future features such as attachments, internal notes, and ticket assignments.

Although the current frontend focuses on replying to tickets, the backend should also support status transitions, categories, and escalation workflows so the platform can evolve without major API redesign.

3.8.15. Data Flow



My assessment

The page completes the operational side of the Platform Owner module. Together with **Monitoring, Emergency Controls, and Developer Tools**, it forms the platform's support and operations suite. One enhancement I'd recommend for the backend is to model support tickets as **conversation threads** instead of single messages. That design will support richer interactions,

attachments, and future integrations (such as email synchronization or assignment to dedicated support agents) without requiring major architectural changes.

3.9 System Monitoring (SAMonitoring.jsx)

Platform Owner Specification Document

Module

Super Admin

Page

SAMonitoring.jsx

3.9.1. Introduction

The **System Monitoring** page provides the Platform Owner with real-time visibility into the operational health of the School Management System platform. It enables continuous monitoring of server performance, system alerts, application logs, and active user activity to facilitate rapid detection and response to operational issues.

3.9.2. Business Objective

The objective of the System Monitoring page is to enable the Platform Owner to:

- Monitor infrastructure health.
- Detect performance degradation.
- View operational alerts.
- Monitor application logs.
- Observe current platform usage.
- Support incident detection and troubleshooting.

The page supports proactive system administration rather than business operations.

3.9.3. Page Overview

The page consists of five major sections.

A. System Status Banner

Displays the current operational status of the platform.

Example:

All Systems Operational

This indicator should reflect the current health of all monitored services.

B. Resource Monitoring

Displays real-time metrics including:

- CPU Usage
- Memory Usage
- Server Uptime
- Active Users

The frontend currently simulates CPU and memory updates every three seconds. In production, these values should come from backend monitoring services.

C. Active Alerts

Displays current platform alerts.

Alert levels include:

- Critical
- Warning
- Information

Each alert contains:

- Alert message
- Severity
- Time generated

D. System Logs

Displays application logs.

Supported log levels:

- INFO
- WARN
- ERROR

The Platform Owner may filter logs by severity.

E. Log Refresh

Allows the Platform Owner to refresh log entries.

3.9.4. Actors / User Roles

Role	Permission
Platform Owner	Full Access
DevOps Engineer (<i>optional</i>)	Configurable
School Administrator	No Access
Teacher	No Access
Student	No Access

Authentication

- JWT Required

Authorization

`monitoring.view`

3.9.5. Functional Requirements

5.1 Platform Health Status

Purpose

Provide an immediate overview of platform availability.

Expected Behaviour

The system shall display one of the following statuses:

- Operational

- Degraded Performance
- Partial Outage
- Major Outage

The displayed status should be derived from monitored backend services rather than hard-coded.

5.2 Resource Monitoring

Purpose

Display real-time infrastructure metrics.

The system shall display:

- CPU Utilization (%)
- Memory Utilization (%)
- Server Uptime
- Active User Count

Expected Behaviour

Metrics should update automatically without requiring a page refresh.

5.3 Active Alerts

Purpose

Notify the Platform Owner of operational events requiring attention.

Supported alert categories include:

- Critical
- Warning
- Information

Each alert shall include:

- Severity
- Description
- Timestamp
- Current status (optional future enhancement)

5.4 System Logs

Purpose

Provide access to operational and application logs.

Each log entry shall include:

- Timestamp
- Log level
- Message

Supported levels:

- INFO
- WARN
- ERROR

5.5 Log Filtering

The Platform Owner shall be able to filter logs by:

- ALL
- INFO
- WARN
- ERROR

Filtering should occur server-side for large log volumes to improve performance.

5.6 Refresh Logs

The Platform Owner shall be able to request the latest log entries.

3.9.6. User Interface Components

Component	Description
System Status Banner	Displays overall platform health
CPU Usage Card	Displays processor utilization
Memory Usage Card	Displays memory utilization
Uptime Card	Displays server uptime

Component	Description
Active Users Card	Displays current active users
Active Alerts Panel	Lists current alerts
Alert Severity Badge	Indicates alert priority
System Logs Console	Displays operational logs
Log Filter Buttons	Filter logs by severity
Refresh Button	Reload latest logs

3.9.7. Business Rules

- Platform status shall accurately reflect the health of monitored services.
- CPU and memory usage shall never exceed 100%.
- Critical alerts shall be displayed before warning and informational alerts.
- Logs shall be ordered from newest to oldest.
- Log filtering shall not modify the stored logs.
- Only authorized users may access monitoring information.
- Monitoring information should be read-only from this interface.

3.9.8. Backend API Specification

Platform Health

Endpoint

GET /api/superadmin/monitoring/status

Response

```
{
  "status": "operational",
  "uptime": 99.8,
  "activeUsers": 1247
}
```

Resource Usage

GET /api/superadmin/monitoring/resources

Response

```
{
  "cpu": 34,
```



```
"memory":67,  
"disk":42,  
"activeUsers":1247  
}
```

Active Alerts

GET /api/superadmin/monitoring/alerts

Response

```
[  
  {  
    "id":1,  
    "severity":"critical",  
    "message":"High memory usage on server-01",  
    "timestamp":"2026-02-22T09:52:00Z"  
  }  
]
```

System Logs

GET /api/superadmin/monitoring/logs

Query Parameters

```
level=  
page=  
limit=  
from=  
to=  
search=
```

Refresh Monitoring Data

Rather than creating a separate refresh endpoint, the frontend can simply re-request the monitoring endpoints above. If real-time updates are required, consider using WebSockets or Server-Sent Events (SSE).

3.9.9. Database Requirements

This page primarily consumes operational data rather than storing business records.

Potential sources include:

Source	Purpose
<code>system_logs</code>	Application logs
<code>alerts</code>	Active platform alerts
Monitoring service	CPU, memory, uptime
Session store	Active user count
Audit logs	Operational history (optional)

3.9.10. Validation Rules

Resource Metrics

- CPU must be between 0 and 100.
- Memory must be between 0 and 100.
- Active users must not be negative.

Alerts

- Severity must be one of: `critical`, `warning`, or `info`.
- Alert messages are required.

Logs

- Log level must be valid.
- Timestamp is required.
- Messages should not be empty.

3.9.11. Security & Permissions

Authentication

- JWT Authentication Required

Authorization

`monitoring.view`

Security Measures

- Restrict monitoring data to authorized personnel.

- Mask sensitive information (e.g., IP addresses or credentials) in logs where appropriate.
- Record access to monitoring dashboards for auditing.
- Use HTTPS for all monitoring traffic.

3.9.12. Error Handling

Error	HTTP Status
Invalid JWT	401 Unauthorized
Permission Denied	403 Forbidden
Monitoring Service Unavailable	503 Service Unavailable
Logs Unavailable	500 Internal Server Error
Invalid Filter	400 Bad Request

The frontend should display meaningful status messages and retain the last known data if temporary monitoring failures occur.

3.9.13. Acceptance Criteria

The page is considered complete when:

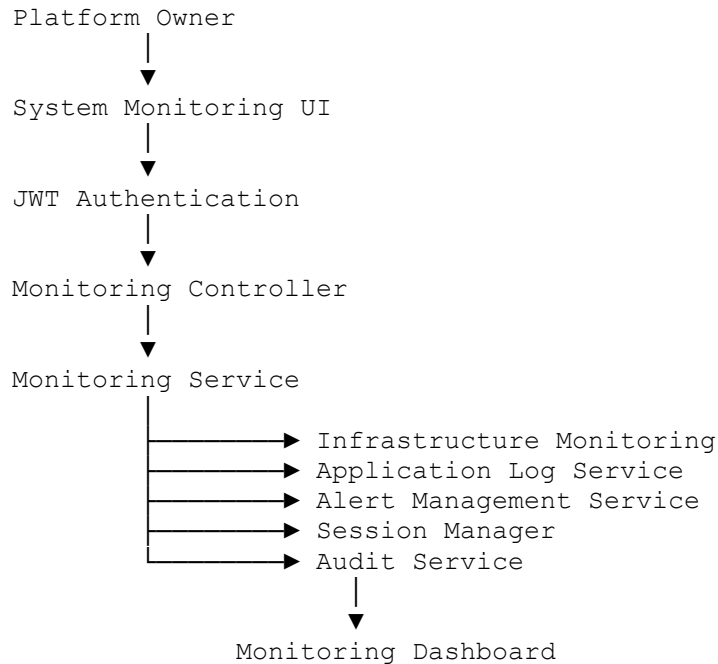
- Platform status displays accurately.
- CPU and memory metrics update correctly.
- Active user count reflects current sessions.
- Alerts display with correct severity indicators.
- Logs are retrieved and filtered successfully.
- Refresh retrieves the latest monitoring data.
- Unauthorized users cannot access monitoring information.

3.9.14. Backend Implementation Notes

The Monitoring page should integrate with a dedicated monitoring or observability service rather than relying solely on application database queries. Metrics such as CPU, memory, and uptime are best collected from infrastructure monitoring tools or the hosting environment, while application logs should be retrieved from a centralized logging system.

Although the frontend currently refreshes CPU and memory using timers, a production implementation would benefit from **WebSockets** or **Server-Sent Events (SSE)** to provide near real-time updates with lower latency and better user experience. Alerts and logs should also support pagination and filtering to accommodate large volumes of operational data.

3.9.15. Data Flow



Backend observation

The frontend simulates CPU and memory utilization with local React state while defining alerts and logs as static arrays. When the backend is implemented, these values should be replaced with live monitoring data. I also recommend using a centralized logging solution and real-time event streaming (WebSockets or SSE) so that the Platform Owner receives updates without manually refreshing the page. This aligns well with the operational nature of the Monitoring module.

3.10 Developer Tools (SADeveloperTools.jsx)

Platform Specification Document

Module

Super Admin

Page

SADeveloper Tools.jsx

3.10.1. Page Information

Item	Description
Module	Super Admin
Page	Developer Tools
Route	<code>/superadmin/developer-tools</code> <i>(based on routing convention)</i>
Authorized Role	Platform Owner
Authentication	JWT Required
Authorization	<code>developer.tools.manage</code>

3.10.2. Overview

The Developer Tools page provides the Platform Owner with utilities for managing API credentials, monitoring API activity, testing endpoints, and operating in a sandbox environment without affecting production data.

This page is intended for platform maintenance, diagnostics, integrations, and development support.

3.10.3. Business Objective

The page shall enable the Platform Owner to:

- Manage API keys
- Search API logs
- Test REST API endpoints
- Enable Sandbox Mode
- Copy API credentials
- Revoke compromised API keys

- Generate new API keys

Unlike Analytics, this page supports operational tasks rather than business reporting.

3.10.4. Access Control

Role	Access
Platform Owner	Full
Developer	Optional (future)
School Admin	No
Teacher	No
Student	No

3.10.5. Functional Requirements

3.10.5.1 Sandbox Mode

The Platform Owner shall be able to

- Enable Sandbox Mode
- Disable Sandbox Mode

When Sandbox Mode is enabled

- No production data shall be modified.
- All write operations shall be simulated.
- API keys displayed in the interface shall be masked.

The frontend currently masks API keys by replacing characters when Sandbox Mode is enabled, indicating that production secrets should not be fully exposed. This behavior should be reinforced by the backend. (Derived from the `sandbox` state and conditional rendering in the component.)

3.10.5.2 API Key Management

The Platform Owner shall be able to

- View API keys
- Generate new API keys
- Copy API keys
- Revoke API keys

Each API Key contains

- Name
- Key
- Date Created
- Last Used
- Status

Backend APIs

GET /api/superadmin/developer/api-keys

POST /api/superadmin/developer/api-keys

DELETE /api/superadmin/developer/api-keys/{id}

Example Response

```
[
  {
    "id":1,
    "name":"Production API Key",
    "status":"active",
    "created":"2026-01-15",
    "lastUsed":"2026-02-22"
  }
]
```

Note: The backend should avoid returning full production secrets after creation. Consider returning a masked value and showing the full key only once when it is generated.

3.10.5.3 API Log Searcher

The Platform Owner shall be able to

- Search logs

- Filter logs
- View API response codes
- View request methods
- View endpoints

The frontend currently demonstrates filtering by search text against mock log entries.

Backend API

GET /api/superadmin/developer/logs

Query Parameters

```
search=  
method=  
status=  
from=  
to=  
page=  
limit=
```

3.10.5.4 API Testing Sandbox

The Platform Owner shall be able to

- Select HTTP Method

Supported methods

- GET
- POST
- PUT
- DELETE

The Platform Owner shall also

- Specify endpoint
- Enter JSON request body
- Execute request
- View JSON response

Backend API

Instead of executing arbitrary requests directly against production, consider routing sandbox requests through a dedicated testing service.

Example:

POST /api/superadmin/developer/sandbox

Request

```
{
  "method": "POST",
  "endpoint": "/api/v1/schools",
  "body": {
    "name": "Test School"
  }
}
```

3.10.6. UI Components

The frontend contains

- Sandbox Toggle
- API Key List
- Copy Button
- Revoke Button
- Generate API Key Button
- Search Box
- API Logs Console
- API Test Form
- JSON Response Viewer

3.10.7. Database Requirements

Potential tables

api_keys

```
id
name
key_hash
status
created_at
last_used_at
```

`created_by`

(Store a hash or securely encrypted secret rather than plain text where possible.)

api_logs

`id`
`method`
`endpoint`
`status_code`
`execution_time`
`ip_address`
`created_at`
`user_id`

developer_requests (optional)

Store sandbox testing history if your team wants an audit trail.

3.10.8. Validation Rules

API Key

- Name required
- Unique

Sandbox Request

- HTTP Method required
- Endpoint required
- Valid JSON body

Search

- Accept partial matches
- Ignore case

3.10.9. Security Requirements

JWT Required

Permission

`developer.tools.manage`

Additional security recommendations:

- Never expose full production API keys after creation.
- Audit all API key generation and revocation events.
- Restrict sandbox execution to authorized roles only.
- Log all developer tool actions for traceability.

3.10.10. Error Handling

Condition	Response
Invalid JWT	401 Unauthorized
Permission Denied	403 Forbidden
Invalid JSON	400 Bad Request
Invalid Endpoint	404 Not Found
Internal Error	500 Internal Server Error

3.10.11. Acceptance Criteria

The page shall be considered complete when

- Sandbox Mode can be enabled and disabled.
- API Keys can be generated.
- API Keys can be revoked.
- API Logs can be searched.
- Sandbox requests return responses.
- Unauthorized users cannot access the page.

3.10.12. Backend Implementation Notes

This page is an **administrative platform utility**, not part of the core school management workflow. Because it handles sensitive operations (API credentials, logs, and request execution), the backend should emphasize security, auditing, and least-privilege access.

In production, API keys should never be stored or displayed in plain text after creation. All actions performed through this page—such as generating keys, revoking keys, toggling sandbox mode, and executing sandbox requests—should be logged for auditing purposes.

3.11 Emergency Controls (SAEmergency.jsx)

Platform Owner Specification Document

Module

Super Admin

Page

SAEmergency.jsx

3.11.1. Introduction

The **Emergency Controls** page provides the Platform Owner with centralized controls for executing critical platform-wide emergency operations. These operations include shutting down the platform, rolling back deployments, forcing user logout, and broadcasting emergency notifications.

Due to the high impact of these operations, the system must enforce strict authentication, authorization, confirmation, and auditing before execution.

3.11.2. Business Objective

The objective of this page is to provide authorized Platform Owners with secure emergency management capabilities that protect the integrity, availability, and security of the School Management System during incidents such as security breaches, service outages, or critical maintenance.

3.11.3. Page Overview

The page consists of the following sections:

A. Emergency Warning Banner

Displays a warning indicating that all actions performed on this page have platform-wide consequences.

B. Emergency Action Cards

The following emergency actions are available:

- Platform Shutdown
- Rollback Last Deployment
- Force Logout All Users
- Emergency Broadcast

Each action is presented as a separate card containing:

- Icon
- Action title
- Description
- Execute button

C. Audit Log

Displays previously executed emergency actions.

Each log entry contains:

- Action performed
- User who executed it
- Date and time
- Reason

D. Confirmation Modal

Critical actions require explicit confirmation before execution.

For example:

Type:

SHUTDOWN

to continue.

The confirmation button remains disabled until the required confirmation text is entered.

E. Emergency Broadcast Modal

Allows the Platform Owner to send an emergency notification.

Fields include:

- Broadcast Title
- Broadcast Message

The message is mandatory before submission.

3.11.4. Actors / User Roles

Role	Permission
Platform Owner	Full Access
System Administrator (Optional)	Configurable
School Administrator	No Access
Teacher	No Access
Student	No Access

Authentication:

- JWT Required

Authorization Permission:

`emergency.manage`

3.11.5. Functional Requirements

3.11.5.1 Platform Shutdown

Purpose

Immediately disable all platform services.

Expected Behaviour

The system shall:

- stop new user requests,
- place the platform into maintenance mode (recommended over a hard shutdown),
- preserve existing data,
- record the shutdown event,
- notify connected users if possible.

Business Rules

- Only Platform Owner may execute.
- Confirmation text **SHUTDOWN** is required.
- Every shutdown must be audited.

3.11.5.2 Rollback Deployment

Purpose

Restore the previous stable version of the application.

Expected Behaviour

The system shall:

- identify the previous deployment,
- initiate rollback,
- verify rollback success,
- record the rollback event.

Business Rules

Confirmation text:

ROLLBACK

is required.

3.11.5.3 Force Logout All Users

Purpose

Invalidate every active session.

Expected Behaviour

The system shall:

- revoke all active JWT refresh tokens,
- invalidate active sessions,
- require all users to authenticate again.

Business Rules

Confirmation text:

FORCE-LOGOUT

must be entered before execution.

3.11.5.4 Emergency Broadcast

Purpose

Notify all platform users during emergencies.

Expected Behaviour

The Platform Owner shall provide:

- Title
- Message

The backend shall distribute the message using configured notification channels such as:

- Email
- SMS
- In-app notifications
- Push notifications (future)

The frontend indicates that a title is optional while the message is mandatory.

3.11.5.5 Audit Logging

Every emergency operation shall create an audit record containing:

- Action
- Executor
- Timestamp
- Reason
- Result (Success/Failure)

- IP Address
- Device Information (recommended)

These logs must not be editable through the UI.

3.11.6. User Interface Components

Component	Description
Danger Zone Banner	Warns about critical actions
Emergency Action Cards	Display available emergency operations
Confirmation Modal	Prevents accidental execution
Broadcast Form	Captures emergency message details
Audit Log	Displays history of emergency operations
Success Notification	Confirms successful execution

3.11.7 Business Rules

- Emergency actions shall only be executed by authorized Platform Owners.
- Confirmation text must exactly match the required keyword.
- Every action must be recorded in the audit log.
- Emergency Broadcast requires a non-empty message.
- Platform Shutdown should preferably place the platform into Maintenance Mode rather than terminating services abruptly.
- Emergency operations must be idempotent where appropriate to prevent accidental repeated execution.

3.11.8 Backend API Specification

Platform Shutdown

Endpoint

POST /api/superadmin/emergency/shutdown

Authentication

JWT Required

Request

```
{
  "confirmation":"SHUTDOWN"
}
```

Response

```
{
  "success":true,
  "message":"Platform entering maintenance mode."
}
```

Rollback Deployment

POST /api/superadmin/emergency/rollback

Force Logout

POST /api/superadmin/emergency/logout-all

Emergency Broadcast

POST /api/superadmin/emergency/broadcast

Request

```
{
  "title":"System Maintenance",
  "message":"The platform will undergo emergency maintenance."
}
```

Audit Logs

GET /api/superadmin/emergency/audit

3.11.9 Database Requirements

emergency_audit_logs

Field	Type
id	UUID

Field	Type
action	VARCHAR
executed_by	UUID
reason	TEXT
status	VARCHAR
ip_address	VARCHAR
user_agent	TEXT
created_at	TIMESTAMP

emergency_broadcasts

Field	Type
id	UUID
title	VARCHAR
message	TEXT
created_by	UUID
sent_at	TIMESTAMP

3.11.10 Validation Rules

Confirmation Validation

- Confirmation text is required.
- It must exactly match the expected keyword.

Broadcast Validation

Title

- Optional
- Maximum 150 characters

Message

- Required
- Maximum 5000 characters

Authorization Validation

Only users with the **Platform Owner** role may execute emergency actions.

3.11.11 Security & Permissions

Authentication

- JWT Authentication Required

Authorization

`emergency.manage`

Security Measures

- Require re-authentication (password or MFA) before executing high-risk actions such as shutdown or rollback.
- Record all emergency operations in immutable audit logs.
- Mask sensitive details where appropriate.
- Protect endpoints with rate limiting to reduce abuse.
- Require HTTPS for all requests.

3.11.12 Error Handling

Error	HTTP Code
Invalid JWT	401
Permission Denied	403
Invalid Confirmation Text	400
Broadcast Message Missing	400
Rollback Failed	500
Shutdown Failed	500

The frontend should display clear error messages without exposing internal server details.

3.11.13 Acceptance Criteria

The page is considered complete when:

- Platform Shutdown requires confirmation and executes successfully.
- Rollback requires confirmation and restores the previous deployment.
- Force Logout invalidates all active user sessions.
- Emergency Broadcast sends messages only when the required message field is provided.
- Audit logs accurately record all emergency actions.
- Unauthorized users are prevented from accessing or executing emergency controls.

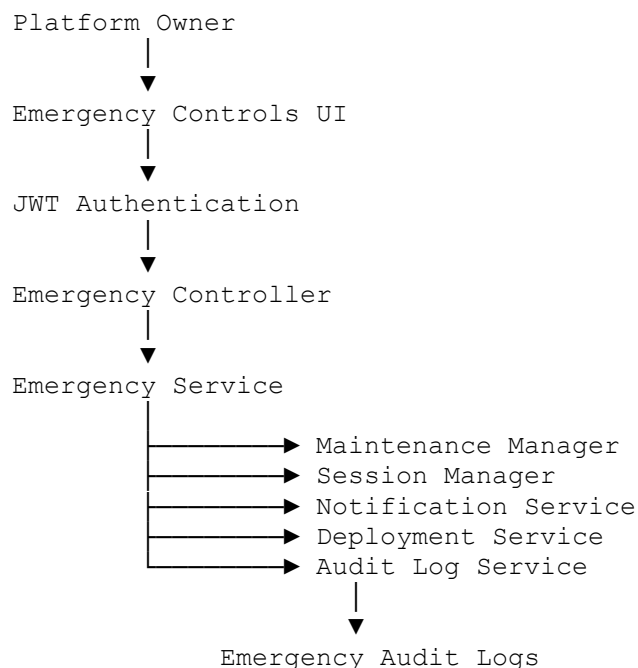
3.11.14 Backend Implementation Notes

This page should be treated as one of the **highest-security modules** in the platform. The frontend already enforces confirmation text before enabling critical actions, but the backend **must never rely solely on client-side validation**. It should independently validate confirmation phrases, verify permissions, and log every operation.

For **Platform Shutdown**, I recommend implementing **Maintenance Mode** instead of powering down services. This approach preserves service availability for administrators while preventing access by regular users and is commonly used in production systems.

For **Force Logout**, if the platform uses JWT authentication, consider storing refresh tokens or maintaining a token blacklist so that all active sessions can be invalidated immediately.

3.11.15 Data Flow



One observation from the frontend

This page has been designed very thoughtfully. The confirmation keywords (`SHUTDOWN`, `ROLLBACK`, `FORCE-LOGOUT`), warning banner, disabled confirmation button until the correct text is entered, and audit log all indicate a strong focus on preventing accidental execution. Those frontend safeguards are excellent, but the backend should enforce the same rules independently to ensure they cannot be bypassed.